```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.feature_selection import VarianceThreshold
from sklearn.linear_model import LogisticRegression
from sklearn.impute import SimpleImputer
from sklearn.metrics import accuracy_score, precision_score, recall_scor
```

Task 1: Identify Prediciton Target

Lecture material: Lecture 1 (Problem Formulation), Lecture 2 (Data Inspection).

• Inspect the dataset and identify which column should be treated as the target variable for this assignment.

• Justify why this column represents the appropriate prediction objective in the context of the marketing campaign.

• Identify at least two other variables that could superficially appear to be valid targets and explain why they should not be treated as the prediction objective.

From our initial inspection of the dataset, we can determine that the target variable for this assessment is column "y". We can come to this conclusion because this column designates whether a customer ended up subscribing to a term deposit after being contacted by the bank. Therefore, we want our model to predict this value. One other variable that may appear to be our target variable is "duration" which measures the duration of the call since we could assume that longer calls mean that the customer is interested and that they are more likely to a successful signup. But this is not a valit target since call duration is only determined after the call ends which defeats the purpose of a predictive model. Another variable that may appear to be the target variable is "poutcome" since its values (failure, success, unknown) seem like they would indicate the success of the campaign. This variable misdirects since actually represents the outcome of a past campaign not the current one.

```
df = pd.read_csv("data/bank-additional-full.csv", sep=';')
df.head()
```

	age	job	marital	education	default	housing	loan	contact	mont
0	56	housemaid	married	basic.4y	no	no	no	telephone	ma
1	57	services	married	high.school	unknown	no	no	telephone	ma
2	37	services	married	high.school	no	yes	no	telephone	ma
3	40	admin.	married	basic.6y	no	no	no	telephone	ma
4	56	services	married	high.school	no	no	yes	telephone	ma

5 rows × 21 columns

Task 2: Data Loading & Exploration

Lecture material: Lecture 1 (Problem Formulation), Lecture 2 (Data Inspection and EDA).

• Load the dataset into a Pandas DataFrame.

• Inspect the structure of the dataset: number of observations, number of features, data types, and basic summary statistics.

• Identify which variables are numerical and which are categorical.

• Analyze the distribution of the target variable and comment on potential class imbalance.

• Detect explicit and implicit missing values (e.g., special categories such as unknown).

• Visualize the distribution of at least: – two numerical variables; and – two categorical variables.

• Identify at least one variable that may require special consideration before modeling (e.g., due to distributional properties, extreme skewness, or availability at prediction time), and briefly justify your reasoning.

From this initial data exploration, we determine that the numerical variables are age, duration, campaign, pdays, previous, emp.var.rate, cons.price.idx, cons.conf.idx, euribor3m and nr.employed. Meanwhile the categorical variables are: contact, day_of_week, default, education, housing, job, loan, marital, month, poutcome and y. there are 41188 entries in total. There is a large imbalance in the target dataset since more than 88% of the dataset have a no label. This will lead to issues with acurracy which will affect the final performance of the model. In the dataset there are no implicit missing values. There are 7 variables that have explicit missingness: default, education, housing, job, loan, marital, pdays. For all of these variables except for pdays missing

values are labled as "unknown". In pdays, we have values labled 999 which indicate that that customer hasn't been contacted before. We will need to adjust this because such a large number disrupts our distributions. One important variable that we have to consider is duration. This variable is very linked to our target variable which causes it to be a risk for data leakage. We drop it since it harms our model more than it helps it.

```
df = pd.read_csv("data/bank-additional-full.csv", sep=';')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 41188 entries, 0 to 41187
Data columns (total 21 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   age             41188 non-null  int64
 1   job             41188 non-null  object
 2   marital         41188 non-null  object
 3   education       41188 non-null  object
 4   default         41188 non-null  object
 5   housing         41188 non-null  object
 6   loan            41188 non-null  object
 7   contact         41188 non-null  object
 8   month           41188 non-null  object
 9   day_of_week     41188 non-null  object
 10  duration        41188 non-null  int64
 11  campaign        41188 non-null  int64
 12  pdays           41188 non-null  int64
 13  previous        41188 non-null  int64
 14  poutcome        41188 non-null  object
 15  emp.var.rate    41188 non-null  float64
 16  cons.price.idx  41188 non-null  float64
 17  cons.conf.idx   41188 non-null  float64
 18  euribor3m       41188 non-null  float64
 19  nr.employed     41188 non-null  float64
 20  y               41188 non-null  object
dtypes: float64(5), int64(5), object(11)
memory usage: 6.6+ MB
```

```
numerical_variables = df.select_dtypes(include=[np.number]).columns.toli
categorical_variables = df.columns.difference(numerical_variables).tolis
print(f"Numerical Variables:\n    {numerical_variables}")
print(f"Categorical Variables:\n    {categorical_variables}")
```

```
Numerical Variables:
    ['age', 'duration', 'campaign', 'pdays', 'previous', 'emp.var.rate',
Categorical Variables:
    ['contact', 'day_of_week', 'default', 'education', 'housing', 'job',
```

```
print(df["y"].value_counts())
```

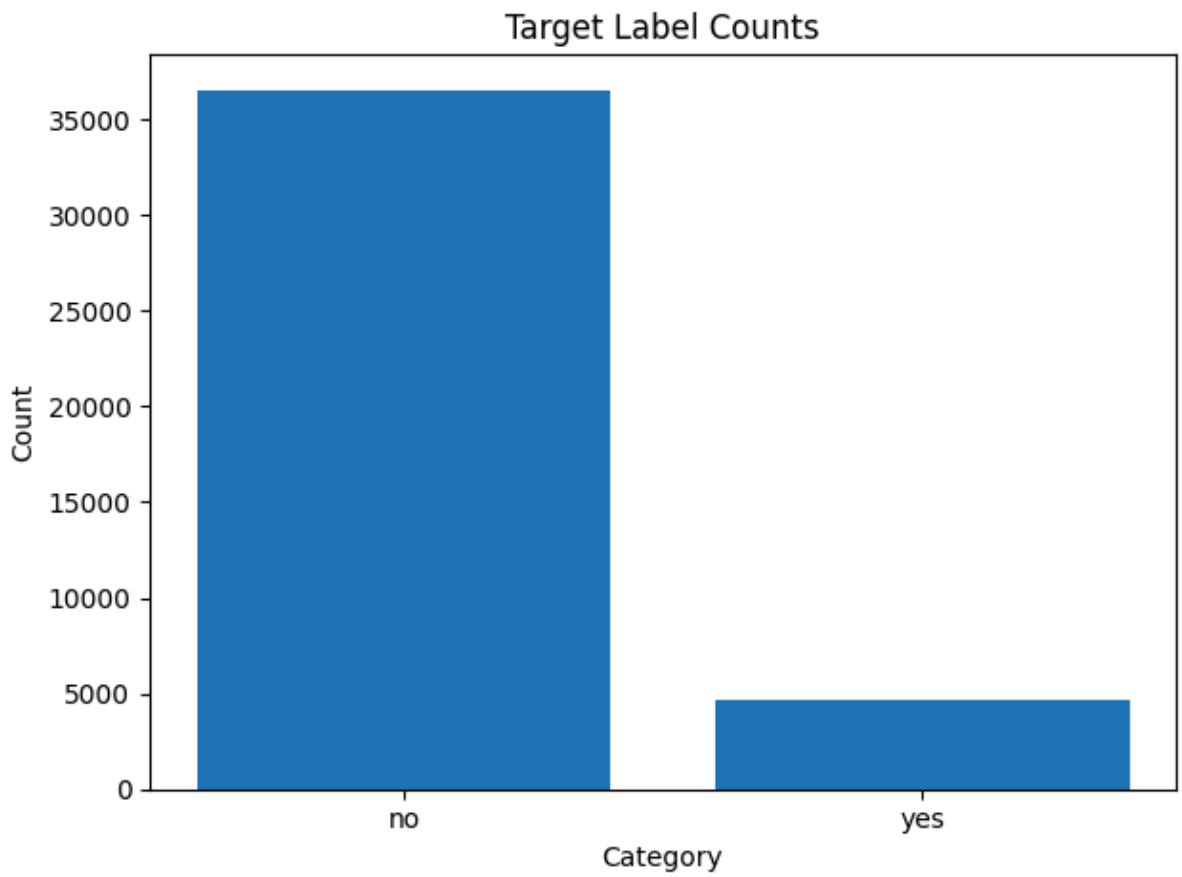
```
y
no     36548
yes     4640
Name: count, dtype: int64
```

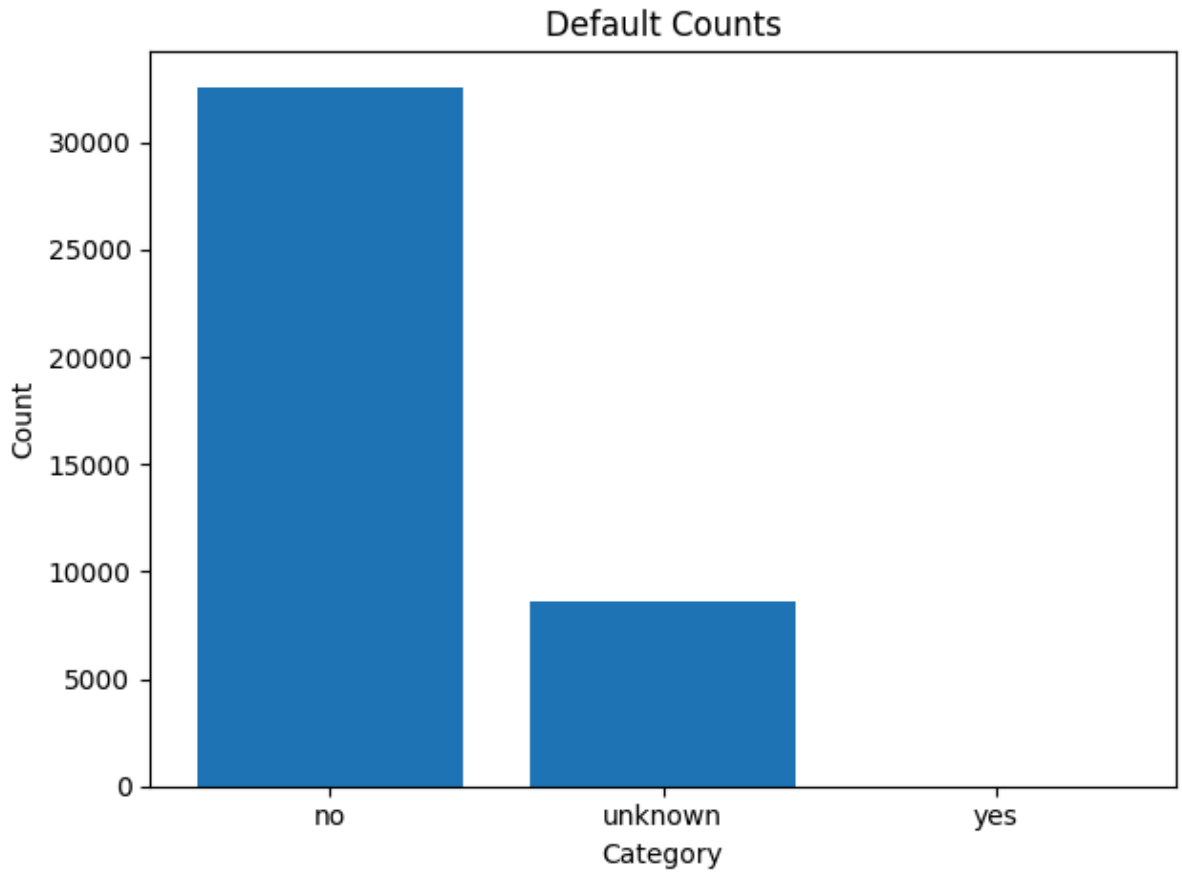
```
print("\nImplicit Missing Values: ")
print(df.isna().sum().sort_values(ascending=False).head(15))


print("\nExplicit Missing Values:")
for cat_column in categorical_variables:
    unknown_count = df[cat_column].value_counts().get("unknown")
    if unknown_count is not None:
        print(f"   {cat_column}: {unknown_count}")
```

```
Implicit Missing Values:
age            0
job            0
marital        0
education      0
default        0
housing        0
loan           0
contact        0
month          0
day_of_week    0
duration       0
campaign       0
pdays          0
previous       0
poutcome       0
dtype: int64

Explicit Missing Values:
   default: 8597
   education: 1731
   housing: 990
   job: 330
   loan: 990
   marital: 80
```

```
# Visualize the distribution of at least:
#    - two numerical variables
#    - two categorical variables.


#   - Two Categorical Variables
```

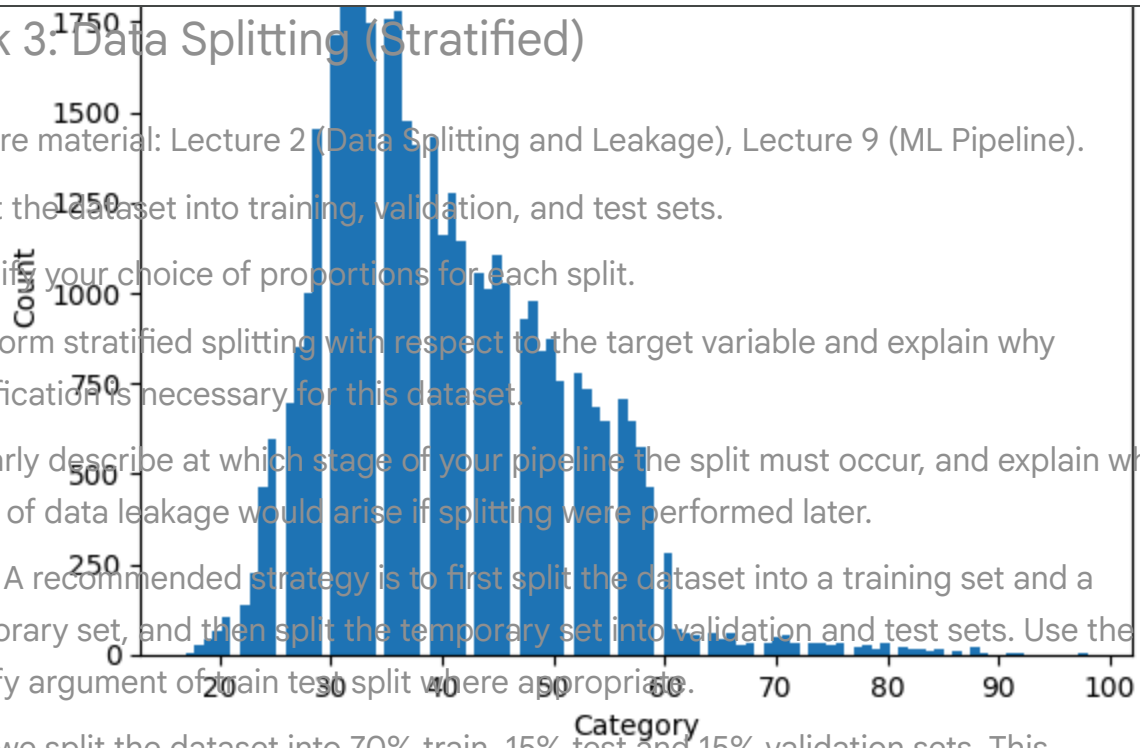
```
counts = df["y"].value_counts()
plt.bar(counts.index.astype(str), counts.values)
plt.xlabel("Category")
plt.ylabel("Count")
plt.title("Target Label Counts")
plt.tight_layout()
plt.show()

counts = df["default"].value_counts()
plt.bar(counts.index.astype(str), counts.values)
plt.xlabel("Category")
plt.ylabel("Count")
plt.title("Default Counts")
plt.tight_layout()
plt.show()


#  - Two Numerical Variable

plt.hist(df["age"], bins=100)
plt.xlabel("Category")
plt.ylabel("Count")
plt.title("Employed Counts")
plt.show()


plt.hist(df["previous"], bins=10)
plt.xlabel("Category")
plt.ylabel("Count")
plt.title("Previous Loans Count")
plt.tight_layout()
plt.show()
```






```
df = df.drop("duration", axis=1)
```

Task 3: Data Splitting (Stratified)



Lecture material: Lecture 2 (Data Splitting and Leakage), Lecture 9 (ML Pipeline).

• Split the dataset into training, validation, and test sets.

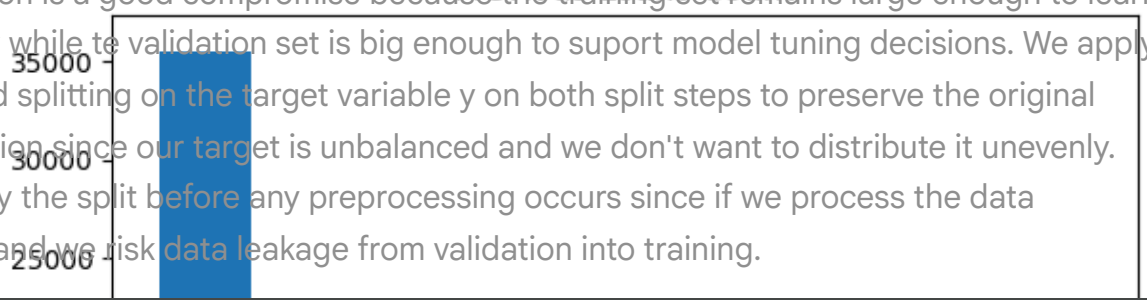
• Justify your choice of proportions for each split.

• Perform stratified splitting with respect to the target variable and explain why stratification is necessary for this dataset.

• Clearly describe at which stage of your pipeline the split must occur, and explain what types of data leakage would arise if splitting were performed later.

Note: A recommended strategy is to first split the dataset into a training set and a temporary set, and then split the temporary set into validation and test sets. Use the stratify argument of train test split where appropriate.

Here we split the dataset into 70% train, 15% test and 15% validation sets. This proportion is a good compromise because the training set remains large enough to learn properly while te validation set is big enough to suport model tuning decisions. We apply stratified splitting on the target variable y on both split steps to preserve the original distribution since our target is unbalanced and we don't want to distribute it unevenly. We apply the split before any preprocessing occurs since if we process the data beforehand we risk data leakage from validation into training.




```
train, test_val = train_test_split(df, test_size=0.3, random_state=42, st
test, val = train_test_split(test_val, test_size=0.5, random_state=42, st


x_train = train.drop("y", axis=1)
y_train = train["y"]

x_test = test.drop("y", axis=1)
y_test = test["y"]

x_val = val.drop("y", axis=1)
y_val = val["y"]
```

Category

Task 4: Missing Value Handling (fit on train)

Lecture material: Lecture 2 (Data Inspection), Lecture 5 (Preprocessing and Pipeline Discipline).

• Identify both explicit missing values (e.g., NaN) and implicit missing values (e.g., categories such as unknown or sentinel numerical values, i.e., values that may represent special codes rather than genuine measurements).

• Quantify the extent of missingness for each affected variable.

• Propose and justify a strategy for handling missing values in each case (e.g., removal, imputation, separate category, indicator variable).

• Clearly state which operations must be fitted using the training set only, and explain why.

Note: Your strategy should distinguish between “data cleaning” decisions (e.g., correcting inconsistent entries) and “modeling” decisions (e.g., whether missingness itself may carry predictive information).

From our code we can determine that there are no explicit missing values. There are 7 variables that have explicit missingness: default, education, housing, job, loan, marital, pdays. For all of these variables except for pdays missing values are labled as "unknown". In pdays, we have sentinel values labled 999 which indicate that that customer hasn't been contacted before. The extent of the missingness is quantified by calculating how many times these values appear in each column. We will be treating unknown as its own category so that the model can learn whether the missingness is informative.For the sentinel value, we replace the 999 with a different code (-1) so that it still provides value without introducing a massive variance we also add a new variable that indicates that the customer has never been contacted. Any operation that learns parameters from data must be fitted on the training set only and then applied unchanged to validation and test sets, to avoid leaking information from validation/test into the preprocessing steps and over-optimistic performance estimates.

```
explicit_missing_counts = df.isna().sum().sort_values(ascending=False)
explicit_missing_counts.head(20)
```

	0
age	0
job	0
marital	0
education	0
default	0
housing	0
loan	0
contact	0
month	0
day_of_week	0
campaign	0
pdays	0
previous	0
poutcome	0
emp.var.rate	0
cons.price.idx	0
cons.conf.idx	0
euribor3m	0
nr.employed	0
y	0

dtype: int64

```
cat_columns = x_train.select_dtypes(include=["object", "category", "stri
for cat_column in cat_columns:
    unknown_count = x_train[cat_column].value_counts().get("unknown")
    if unknown_count is not None:
        print(f"   {cat_column}: {unknown_count}")


sentinel_count = x_train["pdays"].value_counts().get(999)
print(f"   pdays: {sentinel_count}")
```

```
   job: 240
   marital: 58
   education: 1234
```

```
default: 6051
housing: 707
loan: 707
pdays: 27734
```

```
report = pd.DataFrame({
    "explicit_na_count": x_train.isna().sum(),
    "explicit_na_pct": x_train.isna().mean()*100,
})

for c in cat_columns:
    report.loc[c, "implicit_unknown_count"] = (x_train[c].astype(str).st
    report.loc[c, "implicit_unknown_pct"] = report.loc[c, "implicit_unkn

report.loc["pdays", "implicit_999_count"] = (x_train["pdays"]==999).sum(
report.loc["pdays", "implicit_999_pct"] = report.loc["pdays", "implicit_

report.sort_values(by=["explicit_na_count","implicit_unknown_count"], as
```

	explicit_na_count	explicit_na_pct	implicit_unknown_count
default	0	0.0	6051.0
education	0	0.0	1234.0
housing	0	0.0	707.0
loan	0	0.0	707.0
job	0	0.0	240.0
marital	0	0.0	58.0
contact	0	0.0	0.0
month	0	0.0	0.0
day_of_week	0	0.0	0.0
poutcome	0	0.0	0.0
age	0	0.0	NaN
campaign	0	0.0	NaN
pdays	0	0.0	NaN
previous	0	0.0	NaN
emp.var.rate	0	0.0	NaN
cons.price.idx	0	0.0	NaN
cons.conf.idx	0	0.0	NaN
euribor3m	0	0.0	NaN
nr.employed	0	0.0	NaN

```
def add_pdays_features(X):
    X = X.copy()
    X["prev_contacted"] = (X["pdays"] != 999).astype(int)
    X["pdays_clean"] = X["pdays"].where(X["pdays"] != 999, -1)
    return X


x_train = add_pdays_features(x_train)
x_val   = add_pdays_features(x_val)
x_test  = add_pdays_features(x_test)

x_train["pdays_clean"].unique()
```

```
array([-1,  4, 10,  3,  2,  6, 17,  9, 12, 15, 13, 11, 16,  7, 14,  1,
5,
        0,  8, 22, 19, 18, 26, 21, 20, 27, 25])
```

Task 5: Categorical Encoding (Fit on train)

Lecture material: Lecture 4 (Categorical Encoding), Lecture 6 (Linear Models), Lecture 9 (Feature Engineering and Expressiveness).

• Identify all categorical variables in the dataset.

• Distinguish between nominal variables (categories without intrinsic order, e.g., job type) and ordinal variables (categories with a meaningful order, e.g., education level), and justify your classification.

• Select and apply an appropriate encoding strategy for each categorical variable.

• Clearly state which encoders must be fitted on the training set only, and explain why.

• Analyze how encoding changes:

```
– the dimensionality of the dataset;

– the interpretability of model coefficients;

– the types of decision boundaries a linear model can represent.
```

The categorical variables in this dataset are job,
marital, education, default, housing, loan, contact, month, day_of_week, poutcome, and the target y. Most are nominal with education being the only ordinal variable. For this logistic regression, a safe choice is to treat all categoricals as ordinal and use a one-hot encoder on them. Any encoder that learns from data must be fitted on the training set only and then applied back unchanged to the validation and test sets to avoid leaking information. In terms of decision boundries, a linear model on one-hot features can represent separate linear effects for each category but with one-hot encoding it cannot exploit any ordering.

```
cat_columns = x_train.select_dtypes(include=["object", "category", "stri
num_columns = x_train.columns.difference(cat_columns).tolist()

ordinal_columns = ["education"]
nominal_columns = [c for c in cat_columns if c not in ordinal_columns]
```

```
ohe_ct = ColumnTransformer(
    transformers=[
        ("categorical", OneHotEncoder(handle_unknown="ignore", sparse_ou
    ],
    remainder="passthrough",
    verbose_feature_names_out=False
)

x_train_ohe = ohe_ct.fit_transform(x_train)
x_test_ohe = ohe_ct.transform(x_test)
x_val_ohe = ohe_ct.transform(x_val)

ohe_feature_names = ohe_ct.get_feature_names_out()

x_train_ohe = pd.DataFrame(x_train_ohe, columns= ohe_feature_names, inde
x_test_ohe = pd.DataFrame(x_test_ohe, columns= ohe_feature_names, index=
x_val_ohe = pd.DataFrame(x_val_ohe, columns= ohe_feature_names, index=x_
```

Task 6: Feature Scaling (fit on train)

Lecture material: Lecture 5 (Feature Scaling), Lecture 6 (Logistic Regression and Optimization).

• Identify the numerical variables that require scaling.

• Select and apply an appropriate scaling strategy (e.g., standardization or normalization) to those variables.

• Justify your choice of scaling method in the context of Logistic Regression.

• Clearly state which transformations must be fitted on the training set only, and explain why.

• Discuss how feature scaling affects:

```
– gradient-based optimization;

– the magnitude and comparability of model coefficients;

– the interpretation of regularization penalties.
```

All of the numerical variables apart from binary features in this dataset would benefit from scaling. For logistic regression, one of the standard ways of scaling numerical values is through standardization because it puts all predictors on a comparable scale around zero wich stabilizes the optimization and makes the regularization behave more sensibly. The standardization parameters must only be fitted to the training dataset and

then they must be applied unchanged to the test and val data. This is to avoid data leakagefrom those sets into the preprocessing. Proper feature scaling improves gradient-based optimization by making the loss surface more isotropic. it maket the magnitudes of coefficients more comparable since a one unit change in each standarized variable is the same. It also clarigfies the effect of regularization penalties (L1/L2) since the penalty affects each coefficient in roughly the same way.

```
print("Variances of numerical columns:")
for i in num_columns:
    print(f"    {i}: {round(x_train_ohe[i].var(), 2)}")


standardization = ColumnTransformer(
    transformers=[
        ("Standarize", StandardScaler(), num_columns)
    ],
    remainder="passthrough",
    verbose_feature_names_out=False
)

x_train_std = standardization.fit_transform(x_train_ohe)
x_test_std = standardization.transform(x_test_ohe)
x_val_std = standardization.transform(x_val_ohe)


std_feature_names = standardization.get_feature_names_out()

x_train_std = pd.DataFrame(x_train_std, columns= std_feature_names, inde
x_test_std = pd.DataFrame(x_test_std, columns= std_feature_names, index=
x_val_std = pd.DataFrame(x_val_std, columns= std_feature_names, index=x_

print(x_train.head())
```

```
Variances of numerical columns:
    age: 108.03
    campaign: 7.58
    cons.conf.idx: 21.38
    cons.price.idx: 0.33
    emp.var.rate: 2.47
    euribor3m: 3.01
    nr.employed: 5252.32
    pdays: 36091.13
    pdays_clean: 2.4
    prev_contacted: 0.04
    previous: 0.25
       age          job   marital            education  default housing l
7309    50  blue-collar    single             basic.4y       no      no
4169    51     services   married  professional.course  unknown      no
35810   46   technician  divorced  professional.course       no     yes
```

```
9668    46       admin.    single    university.degree       no     yes
34389   25     services    single          high.school       no      no

         contact month day_of_week  ...  pdays  previous     poutcome  \
7309   telephone   may         thu  ...    999         0  nonexistent
4169   telephone   may         mon  ...    999         0  nonexistent
35810   cellular   may         mon  ...    999         0  nonexistent
9668   telephone   jun         mon  ...    999         0  nonexistent
34389   cellular   may         thu  ...    999         0  nonexistent

      emp.var.rate  cons.price.idx  cons.conf.idx  euribor3m  nr.employed
7309           1.1          93.994          -36.4      4.860       5191.0
4169           1.1          93.994          -36.4      4.858       5191.0
35810         -1.8          92.893          -46.2      1.244       5099.1
9668           1.4          94.465          -41.8      4.961       5228.1
34389         -1.8          92.893          -46.2      1.266       5099.1

       prev_contacted  pdays_clean
7309                0           -1
4169                0           -1
35810               0           -1
9668                0           -1
34389               0           -1

[5 rows x 21 columns]
```

Task 7: Feature Selection (fit on train)

Lecture material: Lecture 5 (Feature Selection), Lecture 6 (Linear Models), Lecture 9 (Pipeline Discipline).

• Identify and remove features with very low variance, if any. Justify the criterion used to define “low” variance.

• Identify highly correlated numerical features and decide whether any should be removed. Clearly state the threshold used and justify your decision.

• Discuss whether any features should be removed based on conceptual considerations (e.g., redundancy, availability at prediction time, or risk of data leakage).

• Explain why feature selection must be performed using the training set only.

• Discuss the consequences of performing feature selection on the entire dataset before splitting.

In this dataset, certain numerical features have a non-trivial spread, so a reasonable filter can treat any feature with a variance below a small threshold as a constant and remove it since these features contribute almost nothing to the model. Amongst the remaining numerical values, we can find some highly correlated features. In this situation,

it is reasonable to remove one of the variables from each of the correlated feature pairs in order to reduce multicollinearity. Apart from this, we can also remove other features such as variables that will not be known at prediction time or those that directly encode the target. all such feature sets must be designed and fitted on the training set only. If feature selection is instead performed on the entire dataset before splitting, the model's choice of features would be informed by the patterns present in what will later become the test/val datasets. In practice this will lead to inflated metrics.

```
selector = VarianceThreshold(threshold=0.01)
selector.fit(x_train_std)

x_train_var = selector.fit_transform(x_train_std)
x_test_var = selector.transform(x_test_std)
x_val_var = selector.transform(x_val_std)

var_feature_names = selector.get_feature_names_out()

x_train_var = pd.DataFrame(x_train_var, columns= var_feature_names, inde
x_test_var = pd.DataFrame(x_test_var, columns= var_feature_names, index=
x_val_var = pd.DataFrame(x_val_var, columns= var_feature_names, index=x_
```

```
num_x_train_var = x_train_var[num_columns]


corr_matrix = num_x_train_var.corr().abs()


upper = corr_matrix.where(
    np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)
)

threshold = 0.9
drop = [col for col in upper.columns if any(upper[col] > threshold)]
print("Highly correlated pairs (>|0.9|):")
for col in upper.columns:
    high_corr = upper.index[upper[col] > threshold].tolist()
    if high_corr:
        for other in high_corr:
            print(f"{other} — {col}: {upper.loc[other, col]}")

drop.remove("prev_contacted")

print("\nFeatures chosen to drop:")
print(drop)


x_train_sel_corr = num_x_train_var.drop(columns=drop)
```

```
Highly correlated pairs (>|0.9|):
emp.var.rate — euribor3m: 0.9721916546813714
emp.var.rate — nr.employed: 0.9060245243178423
euribor3m — nr.employed: 0.9448735707557098
pdays — prev_contacted: 0.9999917749253908

Features chosen to drop:
['euribor3m', 'nr.employed']
```

Task 8: Address Class Imbalance (train only)

Quantify the class distribution in the training set and explain why imbalance is or is not a concern for this prediction task.

• Propose a resampling strategy (e.g., random oversampling, SMOTE, or ADASYN). Clearly justify at which stage of the pipeline the resampling step should occur.

• Justify your choice of resampling method in terms of its assumptions and expected effect on the learning algorithm.

• Explain what would happen if resampling were applied before splitting the dataset into training, validation, and test sets. Discuss the implications for model evaluation.